

DJANGO

Основные сведения

Django — это фреймворк для создания веб-приложений на Python.

Простыми словами: Django берет на себя всю рутину, чтобы ты мог сосредоточиться на логике своего приложения.

Назначение файлов в проекте Django:

Процесс создания будет описан после, а сейчас разберём назначение файлов проекта фреймворка Django:

Файлы проекта:

Файл	Назначение
settings.py	Настройки проекта (БД, приложения, безопасность)
urls.py	Главная карта маршрутов (какой URL → какое приложение)
wsgi.py	Точка входа для серверов (как запустить приложение)
asgi.py	Для асинхронных запросов (WebSockets)
__init__.py	Делает папку модулем Python

Файлы приложения:

Файл	Назначение
views.py	Здесь ты пишешь логику (что делать с запросом)
urls.py	Создаешь сам (не создается автоматически) — маршруты приложения
models.py	Здесь описываешь структуру базы данных
admin.py	Настройка админки для твоих моделей
tests.py	Тесты для приложения
apps.py	Конфигурация приложения (почти всегда не трогаешь)

1. HttpResponseRedirect и его виды

HttpResponse — это объект, который Django отправляет браузеру.

```
from django.http import HttpResponseRedirect, JsonResponse, HttpResponseRedirect

# 1. Простой HTML ответ
HttpResponse("<h1>Привет</h1>")

# 2. JSON ответ (для API)
JsonResponse({"name": "Дима", "age": 20})

# 3. Перенаправление (редирект)
HttpResponseRedirect('/')

# или короткая форма:
from django.shortcuts import redirect
redirect('/')
```

Рис. 1 HttpResponseRedirect и его виды

2. Django Template Language — шаблонный язык Django

Синтаксис	Назначение	Пример
<code>{{ variable }}</code>	Вывод переменной	<code>{{ task.title }}</code>
<code>{% tag %}</code>	Логика (циклы, условия)	<code>{% if tasks %}</code>
<code>{{ var filter }}</code>	Фильтры (изменение вывода)	<code>{{ date date:"d.m.Y" }}</code>

Основные теги:

```
<!-- Условие -->
{% if tasks %}
    <p>Задач: {{ tasks|length }}</p>
{% elif tasks_empty %}
    <p>Нет задач</p>
{% else %}
    <p>Что-то другое</p>
{% endif %}

<!-- Цикл -->
{% for task in tasks %}
    <div>{{ task.title }}</div>
{% empty %}
    <p>Список пуст</p>
{% endfor %}

<!-- Комментарий (не виден в HTML) -->
{# это комментарий #}
```

Рис. 2 Основные теги DTL

Основные фильтры:

```
{{ name|upper }}      → ДИМА
{{ name|lower }}      → дима
{{ name|capfirst }}   → Дима
{{ text|truncatechars:10 }} → обрезает до 10 символов
{{ date|date:"d.m.Y" }} → 24.03.2025
{{ task.title|default:"Без названия" }}
```

Рис. 3 Основные фильтры DTL

3. {% csrf_token %} — защита от подделки запросов

CSRF (Cross-Site Request Forgery) — атака, когда злой сайт отправляет запрос от имени пользователя.

Как работает:

- Django вставляет в форму скрытое поле с уникальным токеном;
- При отправке формы Django проверяет этот токен;

- Если токена нет или он неверный — Django блокирует запрос (403).

```
<form method="post">
    {% csrf_token %} <!-- вставляет <input type="hidden" name="csrfmiddlewaretoken" value="..." --> --
    >
    <input type="text" name="name">
    <button>Отправить</button>
</form>
```

Рис. 4 Использование токена csrf

Когда нужен: для всех POST, PUT, DELETE запросов (которые меняют данные).

4. Работа с моделями (CRUD операции)

CRUD — Create, Read, Update, Delete.

```
from tasks.models import Task

# ===== CREATE (создание) =====
# Способ 1
task = Task(title="Купить хлеб", description="В магазине")
task.save()

# Способ 2 (через create)
task = Task.objects.create(title="Купить хлеб", description="В магазине")

# Способ 3 (через словарь)
data = {"title": "Купить хлеб", "description": "В магазине"}
task = Task.objects.create(**data)

# ===== READ (чтение) =====
# Все записи
all_tasks = Task.objects.all()

# Одна запись по id
task = Task.objects.get(id=1) # если нет - ошибка

# Фильтрация
pending = Task.objects.filter(completed=False)

# Первая запись, удовлетворяющая условию
first_pending = Task.objects.filter(completed=False).first()

# Поиск по части названия (like)
search = Task.objects.filter(title__contains="хлеб")

# Сортировка
latest = Task.objects.all().order_by('-created_at') # - по убыванию

# Количество записей
count = Task.objects.filter(completed=False).count()

# ===== UPDATE (обновление) =====
task = Task.objects.get(id=1)
task.title = "Новое название"
task.save() # важно!

# Множественное обновление
Task.objects.filter(completed=False).update(completed=True)

# ===== DELETE (удаление) =====
task = Task.objects.get(id=1)
task.delete()

# Множественное удаление
Task.objects.filter(completed=True).delete()
```

Рис. 5 Работа с моделями

5. Работа с запросами (request)

```
def my_view(request):
    # GET-параметры: /page?name=Дима&age=20
    name = request.GET.get('name') # 'Дима'
    age = request.GET.get('age') # '20'
    age_int = request.GET.get('age', 0) # с значением по умолчанию

    # POST-параметры (из формы)
    name = request.POST.get('name')
    description = request.POST.get('description')

    # Информация о пользователе
    method = request.method # 'GET', 'POST', 'PUT', 'DELETE'
    path = request.path # '/addTask/'

    # Проверка метода
    if request.method == 'POST':
        # обрабатываем POST
    else:
        # показываем форму
```

Рис. 6 Работа с запросами

6. Формы (HTML)

```
<!-- GET форма (параметры в URL) -->
<form action="/search/" method="get">
    <input type="text" name="q">
    <button>Искать</button>
</form>

<!-- Результат: /search/?q=текст -->

<!-- POST форма (данные в теле запроса) -->
<form action="/addTask/" method="post">
    {% csrf_token %}
    <input type="text" name="name">
    <input type="text" name="description">
    <button>Добавить</button>
</form>
```

Рис. 7 Формы

7. Перенаправление (redirect)

```
from django.shortcuts import redirect

def addTask(request):
    # ... создаем задачу
    return redirect('/')           # на корень
    return redirect('home')       # по имени маршрута
    return redirect('tasks:home') # с пространством имен (app_name:name)
```

Рис. 8 Перенаправление

Разница GET и POST:

GET	POST
Параметры в URL	Данные в теле запроса
Можно сохранить в закладки	Не сохраняется
Ограниченная длина	Практически без ограничений
Для поиска и фильтрации	Для форм, добавления, изменения

9. Типичные ошибки и их решение

Ошибка	Причина	Решение
403 Forbidden	Нет {% csrf_token %}	Добавь в форму
404 Not Found	Неправильный URL	Проверь urls.py
OperationalError: no such table	Не выполнил миграции	python manage.py migrate
AttributeError: 'None-Type' object has no attribute 'xxx'	request.POST['key'] не существует	Используй request.POST.get('key')

Создание и подготовка папки проекта

Для начала работы с фреймворком Django нужно подготовить папку для проекта, а именно создать виртуальное окружение, установить и создать Django проект.

Для создания виртуального окружения в ранее созданной папке прописываем python (если на компьютере версий несколько, а нужна конкретная, то вместо одного слова python указываем полный путь python.exe нужной версии). Далее активируем окружение командой `venv\Scripts\activate`

```
PS D:\Dev\VSCode\Projects\django_learning> python -m venv venv
PS D:\Dev\VSCode\Projects\django_learning> d:\Dev\VSCode\Projects\django_learning\venv\Scripts\activate
```

Рис. 9 Создание и активация виртуального окружения

Далее устанавливаем framework Django:

```
(venv) PS D:\Dev\VSCode\Projects\django_learning> pip install Django
```

Рис. 10 Установка Django

Теперь создаём проект Django (точка в конце означает создать в текущей папке):

```
(venv) PS D:\Dev\VSCode\Projects\django_learning> django-admin startproject mytodo .
```

Рис. 11 Создание проекта Django

Проверим что всё работает, введём команду «`python manage.py runserver`» и откроем браузер по адресу «`http://127.0.0.1:8000/`», если все работает, то вы увидите сообщение "The install worked successfully! Congratulations!"

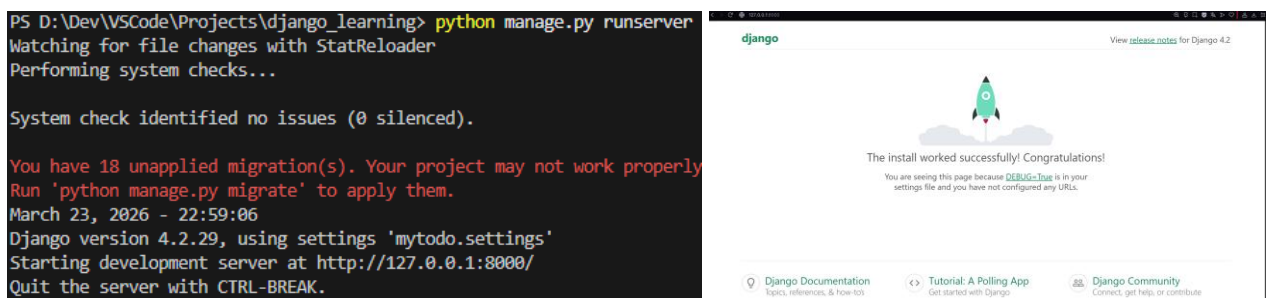


Рис. 12 Проверка работоспособности

Дальше можем создать приложение, делается это командой «`python manage.py startapp tasks`» (название нашего приложения).

```
(venv) PS D:\Dev\VSCode\Projects\django_learning> python manage.py startapp tasks
```

Рис. 13 Создание приложения

Структура проекта будет выглядеть так:

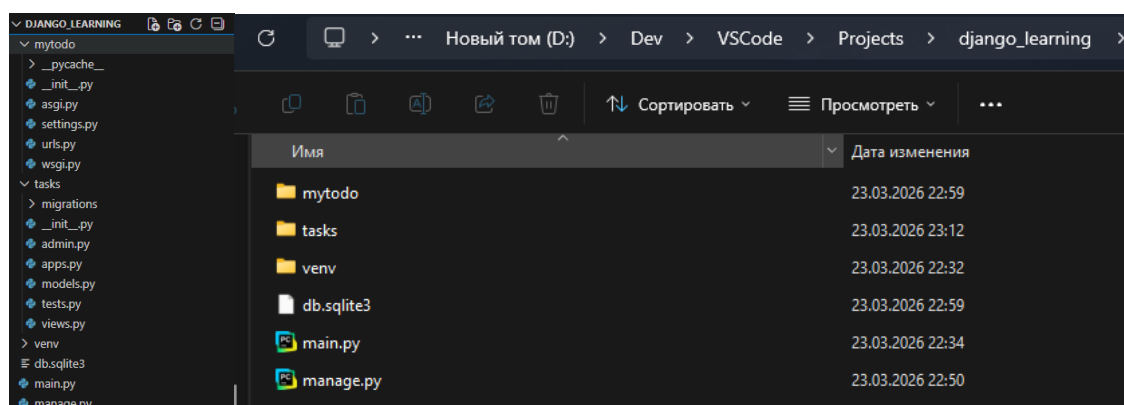


Рис. 14 Вид структуры проекта в среде VS code и проводнике windows

Теперь зарегистрируем наше приложение, для этого откроем mytodo/settings.py, найдём installed_apps и добавим:

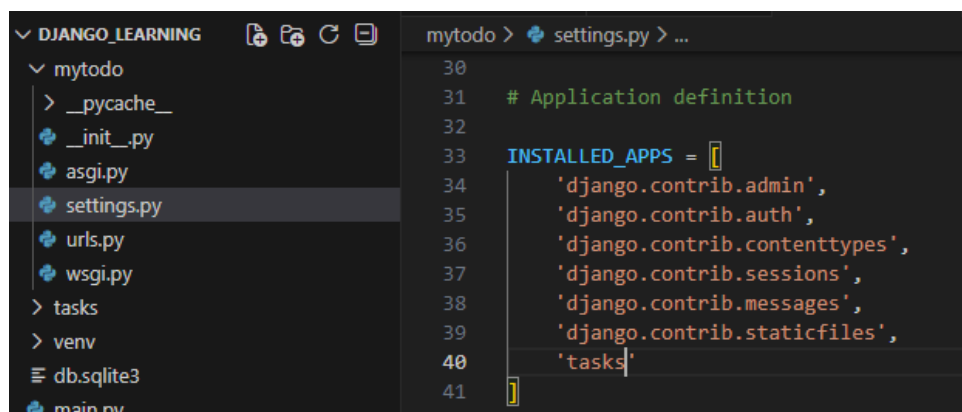


Рис. 15 Регистрация приложения

Создадим нашу первую страницу

Создаётся это по такому принципу, как мы уже знаем, есть файлы views и urls. В views мы напишем, как будет выглядеть эта страница, а в urls – на какой url будет наш сайт возвращать эту страницу.

Сначала сделаем вид страницы, а именно то, как она будет выглядеть в ответ на http запрос, для этого в tasks/views.py и напишем:

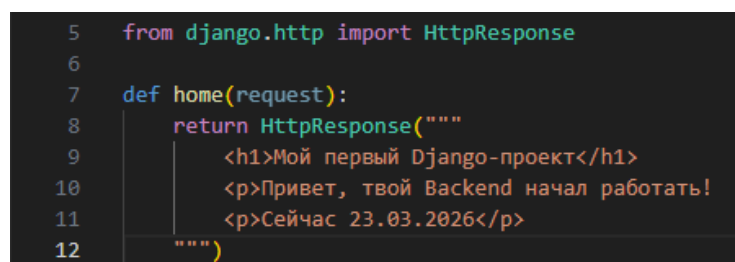


Рис. 16 Определение вида страницы

Затем нам нужно указать, на какой url будет возвращаться данная страница, для этого создадим файл `tasks/urls.py` и запишем туда:

```
tasks > urls.py > ...
1  from django.urls import path
2  from . import views
3
4  urlpatterns = [
5      path('', views.home, name='home'),
6  ]
```

Рис. 17 Ответ на url

На рис. 9 видно, что она будет корневой, путь к её виду `views.home`, а её обозначение «home».

Затем в главной папке `urls` проекта подключим `urls` приложения. Для этого откроем `mytodo/urls.py` и отредактируем его:

```
17  from django.contrib import admin
18  from django.urls import path, include
19
20  urlpatterns = [
21      path('admin/', admin.site.urls),
22      path('', include('tasks.urls')),
23  ]
```

Рис. 18 Указание url

Так же можно сделать проще, указать на вид напрямую из главной папки `urls`:

```
from tasks import views

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', views.home, name='home'),
]
```

Рис. 19 Более простой метод

Но из соображений чистоты кода и понятности советуется применять способ номер 1.

Запустим сервер, откроем страницу и посмотрим результат:



Рис. 20 Результат создания страницы

Динамические URL и шаблоны

Сейчас мы сделаем страницу, которая приветствует пользователя по имени из URL. В `tasks.views`:

```
20 def hello(request, name):
21     return HttpResponse(f"""
22         <h1>Привет, {name}!</h1>
23         <p>Рады тебя видеть на нашем сайте.</p>
24         <p><a href="/">Вернуться на главную</a></p>
25     """)
```

Рис. 21 Вид динамической страницы

Теперь сделаем новый маршрут в `tasks.urls`, в основном `urls` менять ничего не нужно, там и так указан путь к нашему списку адресов из приложения `tasks`, благодаря первому способу.

```
4 urlpatterns = [
5     path('', views.home, name='home'),
6     path('hello/<str:name>/', views.hello, name='hello')
7 ]
```

Рис. 22 Новый маршрут

Что значит `'hello/<str:name>/'`?

- `hello/` — начало адреса;
- `<str:name>` — динамическая часть, которая будет передана в `view` как параметр `name`;
- `/` — закрывающий слэш.

Теперь проверяем, перейдя по адресу <http://127.0.0.1:8000/hello/ТвоёИмя/>:

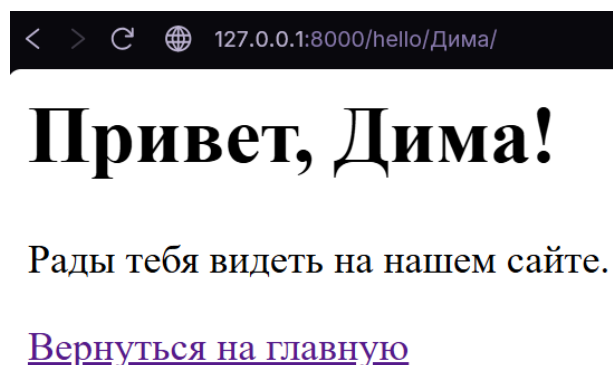


Рис. 23 Результат

Шаблоны (отдельные HTML-файлы)

Писать HTML внутри Python — неудобно. Django умеет работать с отдельными HTML-файлами. Создадим в tasks папку с шаблонами и там напишем тот же вид страниц, но уже в формате файлов html.

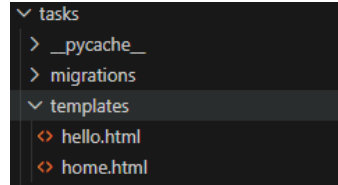


Рис. 24 Структура файлов

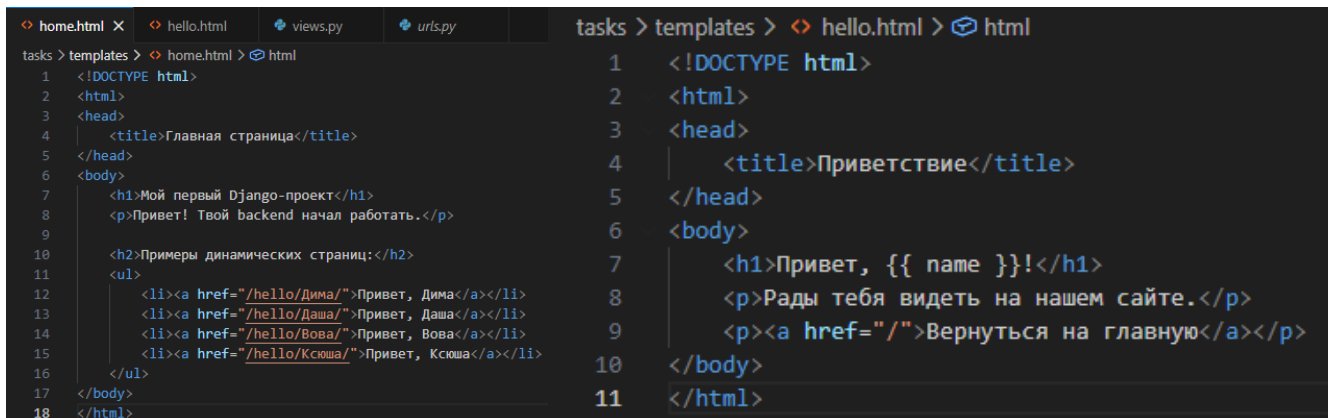


Рис. 25 Html файлы с видами страниц

Обрати внимание: `{{ name }}` - это переменная, которая придет из view.

Обновим наш tasks/view.py для работы с шаблонами.

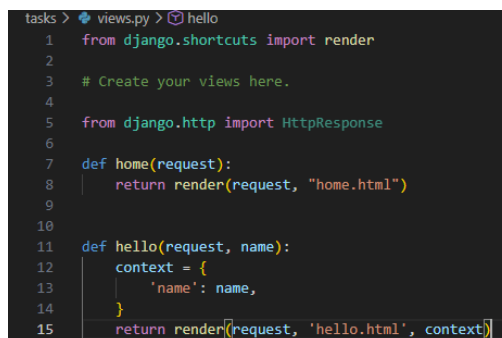


Рис. 26 Изменённый файл с представлениями

Результат:

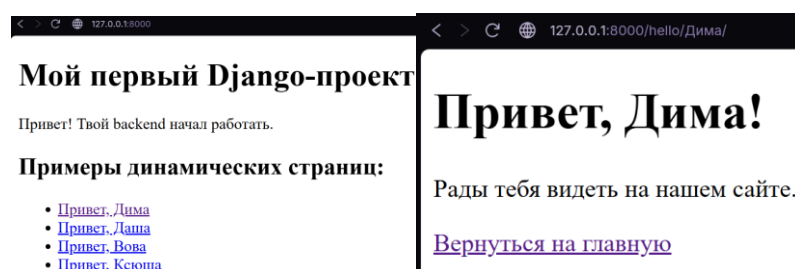


Рис. 27 Результат

Передача нескольких параметров

Создадим страницу, которая показывает информацию о пользователе:

То есть опять по новой, создадим представление, вид и маршрут страницы:

```
18 def user_profile(request, username, age):
19     context = {
20         'username': username,
21         'age': age,
22         'is_adult': age >= 18,
23     }
24     return render(request, 'profile.html', context)
```

Рис. 28 Представление новой страницы

В маршруте видно, что аргументы передаются также через слэши.

```
4 urlpatterns = [
5     path('', views.home, name='home'),
6     path('hello/<str:name>/', views.hello, name='hello'),
7     path('user/<str:username>/<int:age>/', views.user_profile, name='profile'),
8 ]
```

Рис. 29 Маршрут новой страницы

```
tasks > templates > < profile.html > html
1 <!DOCTYPE html>
2 <html>
3 <head>
4     <title>Профиль пользователя</title>
5 </head>
6 <body>
7     <h1>Профиль: {{ username }}</h1>
8     <p>Возраст: {{ age }} лет</p>
9
10     {% if is_adult %}
11         <p style="color: green;">Пользователь совершеннолетний</p>
12     {% else %}
13         <p style="color: red;">Пользователь несовершеннолетний</p>
14     {% endif %}
15
16     <p><a href="/">На главную</a></p>
17 </body>
18 </html>
```

Рис. 30 Вид новой страницы

Теперь проверим, введём в адресную строку:

<http://127.0.0.1:8000/user/Дима/25/>

< > ↻ 127.0.0.1:8000/user/Дима/25/

Профиль: Дима

Возраст: 25 лет

Пользователь совершеннолетний

[На главную](#)

Рис. 31 Результат

Модели

Концепция	Что это
Model	Python-класс, описывающий таблицу в БД
Migration	Файл с описанием изменений в БД
makemigrations	Создает файлы миграций
migrate	Применяет миграции к БД
objects.all()	Получает все записи из таблицы
objects.filter()	Получает записи по условию
Admin panel	Готовый интерфейс для управления данными
@admin.register	Регистрирует модель в админке

Модель – это пайтон класс, который описывает таблицу в базе данных. Создадим класс Task (задача) для нашего будущего списка дел. Как видно на рисунке 24, делается это просто, создается поле с его описанием в виде типа данных и его названия.

```

6 class Task(models.Model):
7     """Модель задачи"""
8     title = models.CharField(max_length=200, verbose_name="Название")
9     description = models.TextField(blank=True, verbose_name="Описание")
10    completed = models.BooleanField(default=False, verbose_name="Выполнено")
11    created_at = models.DateTimeField(auto_now_add=True, verbose_name="Дата создания")
12
13    def __str__(self):
14        return self.title
15
16    class Meta:
17        verbose_name = "Задача"
18        verbose_name_plural = "Задачи"
19        ordering = ['-created_at'] # сортировка по дате (сначала новые)

```

Рис. 32 Содержание файла models

Применим миграции:

```

(venv) PS D:\Dev\VSCode\Projects\django_learning> python manage.py makemigrations
No changes detected
(venv) PS D:\Dev\VSCode\Projects\django_learning> python manage.py migrate

```

Рис. 33 Применение миграций

Вход в админку

Теперь проверим в админке, создав суперпользователя:

```

(venv) PS D:\Dev\VSCode\Projects\django_learning> python manage.py createsuperuser
Username (leave blank to use 'dmitr'): dima
Email address:
Password:
Password (again):

```

Рис. 34 Создание супер пользователя

Зарегистрируем модель в tasks/admin.py:

```
tasks > admin.py > TaskAdmin
1  from django.contrib import admin
2  from .models import Task
3
4  # Register your models here.
5  @admin.register(Task)
6  class TaskAdmin(admin.ModelAdmin):
7      list_display = ('title', 'completed', 'created_at')
8      list_filter = ('completed', 'created_at')
9      search_fields = ('title',)
```

Рис. 35 Регистрация модели

Запустим сервер и зайдем в админку, где создадим пару задач:

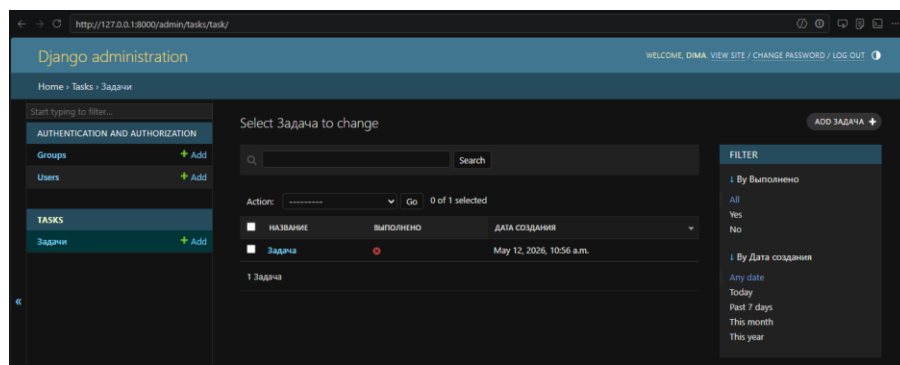


Рис. 36 Запущенная админка

Вывод задач на страницу

Теперь осуществим вывод задач на страницу, для этого обновим наши представление в модуле tasks.

```
5 from django.http import HttpResponse
6 from .models import Task
7
8 def home(request):
9     '''Главная страница со списком задач'''
10    tasks = Task.objects.all()
11    context = {
12        'tasks': tasks,
13        'tasks_count': tasks.count()
14    }
15    return render(request, "tasks/home.html", context)
```

Рис. 37 Обновленное представление

После обновим шаблон страницы:

```
tasks > templates > tasks > home.html > html
2 <html>
3 <head>
4
5 <style>
6     body { font-family: Arial, sans-serif; max-width: 800px; margin: 0 auto; padding: 20px; }
7     .task { border-bottom: 1px solid #eee; padding: 10px 0; }
8     .task.completed { text-decoration: line-through; color: #888; }
9     .task h3 { margin: 0 0 5px 0; }
10    .task p { margin: 0; font-size: 14px; color: #555; }
11    .task small { font-size: 12px; color: #999; }
12    .status-badge { display: inline-block; padding: 2px 8px; border-radius: 4px; font-size: 12px; }
13    .status-completed { background: #d4edda; color: #155724; }
14    .status-pending { background: #f8d7da; color: #721c24; }
15 </style>
16 </head>
17 <body>
18     <h1>Мои задачи</h1>
19     <p>Всего задач: {{ tasks_count }}</p>
20
21     {% if tasks %}
22         {% for task in tasks %}
23             <div class="task {% if task.completed %}completed{% endif %}">
24                 <h3>
25                     {{ task.title }}
26                     {% if task.completed %}
27                         <span class="status-badge status-completed">✓ Выполнено</span>
28                     {% else %}
29                         <span class="status-badge status-pending">o В процессе</span>
30                     {% endif %}
31                 </h3>
32                 {% if task.description %}
33                     <p>{{ task.description }}</p>
34                 {% endif %}
35                 <small>Создано: {{ task.created_at|date:"d.m.Y H:i" }}</small>
36             </div>
37         {% endfor %}
38     {% else %}
39         <p>Пока нет задач. Создайте первую задачу в админке!</p>
40     {% endif %}
41
42     <hr>
43     <h2>Динамические страницы:</h2>
44     <ul>
45         <li><a href="/hello/Дима/">Привет, Дима</a></li>
46         <li><a href="/user/Дима/25/">Профиль Димы</a></li>
47     </ul>
48     <p><a href="/admin/">Админка</a> (создавай задачи)</p>
49 </body>
50 </html>
```

Рис. 38 Обновленный шаблон страницы

Создание задач

Чтобы добавить задачу со страницы добавим контейнер сбора данных и пропишем ему отправку на url/addTask/.

```
18 <h1>Создать задачу</h1>
19 <form action="/addTask/" method="post">{% csrf_token %}
20     Название: <input type="text" name="name"><br>
21     Описание: <input type="text" name="description">
22     <input type="submit" value="Добавить новое дело">
23 </form>
```

Рис. 39 Форма сбора данных

Далее в представлениях создадим функцию обработчик, которая получает данные, добавляет их в базу и возвращает домашнюю страницу.

```
34 def addTask(request):
35     task_name = request.POST['name']
36     desc = request.POST['description']
37     new_task = Task(title=task_name, description=desc)
38     new_task.save()
39     return HttpResponseRedirect('/')
```

Рис. 40 Обработка запроса

Обновляем url.py

```
4 urlpatterns = [
5     path('', views.home, name='home'),
6     path('hello/<str:name>/', views.hello, name='hello'),
7     path('user/<str:username>/<int:age>/', views.user_profile, name='profile'),
8     path('addTask/', views.addTask, name='add_task')
9 ]
```

Рис. 41 Добавление обработчика в url.py

Готово, наблюдаем результат

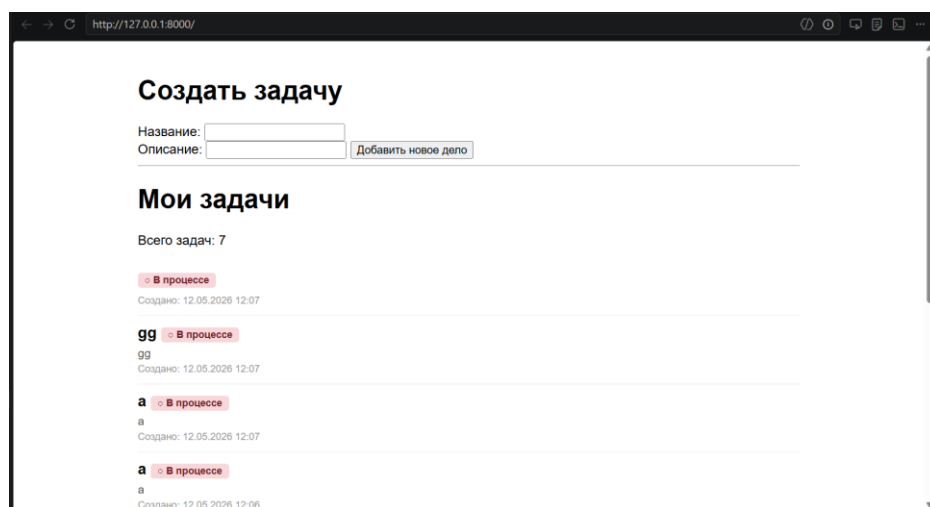


Рис. 42 Вид страницы

Фильтрация задач

Добавим возможность показывать только невыполненные задачи.

В `tasks/views.py` добавим новую функцию:

```

41  def pending_tasks(request):
42      """Страница с невыполненными задачами"""
43      tasks = Task.objects.filter(completed=False)
44      context = {
45          'tasks': tasks,
46          'tasks_count': tasks.count(),
47          'title': 'Невыполненные задачи',
48      }
49      return render(request, 'tasks/task_list.html', context)
50
51  def completed_tasks(request):
52      """Страница с выполненными задачами"""
53      tasks = Task.objects.filter(completed=True)
54      context = {
55          'tasks': tasks,
56          'tasks_count': tasks.count(),
57          'title': 'Выполненные задачи',
58      }
59      return render(request, 'tasks/task_list.html', context)

```

Рис. 43 Новые функции

В `tasks/urls.py` добавим маршруты:

```
4 urlpatterns = [
5     path('', views.home, name='home'),
6     path('hello/<str:name>/', views.hello, name='hello'),
7     path('user/<str:username>/<int:age>/', views.user_profile, name='profile'),
8     path('addTask/', views.addTask, name='add_task'),
9     path('tasks/pending/', views.pending_tasks, name='pending'),
10    path('tasks/completed/', views.completed_tasks, name='completed'),
11 ]
```

Рис. 44 Новые маршруты

Создадим общий шаблон `tasks/templates/tasks/task_list.html`:

```
tasks > templates > tasks > tasl_list.html > html
1 <!DOCTYPE html>
2 <html>
3 <head>
4 <title>{{ title }}</title>
5 <style>
6 body { font-family: Arial, sans-serif; max-width: 800px; margin: 0 auto; padding: 20px; }
7 .task { border-bottom: 1px solid #ccc; padding: 10px 0; }
8 .task.completed { text-decoration: line-through; color: #888; }
9 .task h3 { margin: 0 0 5px 0; }
10 .task p { margin: 0; font-size: 14px; color: #555; }
11 .task small { font-size: 12px; color: #999; }
12 .nav { margin-bottom: 20px; }
13 .nav a { margin-right: 15px; }
14 .status-badge { display: inline-block; padding: 2px 8px; border-radius: 4px; font-size: 12px; }
15 .status-completed { background: #d4edda; color: #155724; }
16 .status-pending { background: #f8d7da; color: #721c24; }
17 </style>
18 </head>
19 <body>
20 <div class="nav">
21 <a href="/"/>Все задачи</a>
22 <a href="/tasks/pending/">Невыполненные</a>
23 <a href="/tasks/completed/">Выполненные</a>
24 <a href="/admin/">Админка</a>
25 </div>
26
27 <h1>{{ title }}</h1>
28 <p>Всего задач: {{ tasks_count }}</p>
29
30 % if tasks %
31 % for task in tasks %
32 <div class="task {{ if task.completed %>completed</if %>">
33 <h3>
34 {{ task.title }}
35 % if task.completed %
36 <span class="status-badge status-completed">✓ Выполнено</span>
37 % else %
38 <span class="status-badge status-pending">○ В процессе</span>
39 % endif %
40 </h3>
41 % if task.description %
42 <p>{{ task.description }}</p>
43 % endif %
44 <small>Создано: {{ task.created_at|date:"d.m.Y H:i" }}</small>
45 </div>
46 % endfor %
47 % else %
48 <p>Нет задач в этой категории.</p>
49 % endif %
50 </body>
51 </html>
```

Рис. 45 Общий шаблон

Обновим home.html — добавь ссылки на фильтры. Вставь в начало <body> после <h1>:

```
17  <body>
18      <div class="nav">
19          <a href="/">Все задачи</a>
20          <a href="/tasks/pending/">Невыполненные</a>
21          <a href="/tasks/completed/">Выполненные</a>
22          <a href="/admin/">Админка</a>
23      </div>
```

Рис. 46 Ссылки на фильтры

Редактирование и удаление

Для редактирования и удаления на странице списка дел были добавлены три формы — на удаление, на изменение и для отметки выполненной задачи. Для отметки и удаления было создана простая форма с соответствующим http запросом на возврат id записи.

```
<form action="/completeTask/" method="post">
  {% csrf_token %}
  <input type="hidden" name="task_id" value="{{ task.id }}">
  {% if task.completed %}
  |   <button type="submit">Отметить не выполненное</button>
  {% else %}
  |   <button type="submit">Отметить как выполненное</button>
  {% endif %}
</form>
<button onclick="openEditModal('{{ task.id }}', '{{ task.title }}', '{{ task.description }}')">
  Редактировать
</button>
<form action="/deleteTask/" method="post">
  {% csrf_token %}
  <input type="hidden" name="task_id" value="{{ task.id }}">
  <button type="submit">Удалить</button>
</form>
```

Рис. 47 Редактирование, удаление и отметка в html шаблоне

Для редактирования же использовалась идея по сложнее, а именно краткий скрипт на модальное окно.

```
<div id="myModal" class="modal">
  <div class="modal-content">
    <span class="close-btn" onclick="closeModal()">&times;</span>
    <h3>Редактировать задачу</h3>
    <form action="/editTask/" method="post">
      {% csrf_token %}
      <input type="hidden" name="task_id" id="modal_task_id">
      Название: <br>
      <input type="text" name="name" id="modal_name"><br><br>
      Описание: <br>
      <input type="text" name="description" id="modal_desc"><br><br>
      <button type="submit">Сохранить</button>
    </form>
  </div>
</div>

<script>
function openEditModal(id, title, desc) {
  document.getElementById("modal_task_id").value = id;
  document.getElementById("modal_name").value = title;
  document.getElementById("modal_desc").value = desc;
  document.getElementById("myModal").style.display = "block";
}
function closeModal() {
  document.getElementById("myModal").style.display = "none";
}
</script>
```

Рис. 48 Скрипт на модальное окно

Так же были написаны обработчики в файле tasks/views.py

```
61 def complete_task(request):
62     task_id = request.POST.get('task_id')
63     task = Task.objects.get(id=task_id)
64     if task.completed:
65         task.completed = "False"
66         task.save()
67     else:
68         task.completed = "True"
69         task.save()
70     return HttpResponseRedirect('/')
71
72 def delete_task(request):
73     task_id = request.POST.get('task_id')
74     if task_id:
75         task = Task.objects.get(id=task_id)
76         task.delete()
77     return HttpResponseRedirect('/')
78
79 def edit_task(request):
80     task_id = request.POST.get('task_id')
81     task_name = request.POST.get('name')
82     desc = request.POST.get('description')
83     task = Task.objects.get(id=task_id)
84     if task_name:
85         task.title = task_name
86     task.description = desc
87     task.save()
88     return HttpResponseRedirect('/')
```

Рис. 49 Обработчики

Обновляем tasks/urls.py, запускаем сервер и смотрим на результат:

The screenshot shows a web application interface. At the top, there are links: [Все задачи](#), [Невыполненные](#), [Выполненные](#), and [Админка](#). Below these is a section titled "Создать задачу" (Create task) with input fields for "Название:" (Name) and "Описание:" (Description), and a button "Добавить новое дело" (Add new case). Below this is a section titled "Мои задачи" (My tasks). It shows "Всего задач: 2" (Total tasks: 2). The first task is titled "Я удивлен" (I am surprised) with a status "В процессе" (In progress). It has a description "Что заработало" (What worked) and buttons "Отметить как выполненное" (Mark as completed), "Редактировать" (Edit), and "Удалить" (Delete). It was created on "12.05.2026 17:28". The second task is titled "Новая задача" (New task) with a status "В процессе" (In progress). It has a description "Ура" (Hooray) and buttons "Отметить как выполненное" (Mark as completed), "Редактировать" (Edit), and "Удалить" (Delete). It was created on "12.05.2026 17:27".

Рис. 50 Результат

Detail views

Сделаем страницу по просмотру деталей о задаче. Для этого опять создадим функцию обработчик во views.py, которая принимает обязательный объект запроса request и переменную task_id из URL-адреса, после находит и извлекает одну запись из таблицы Task, у которой поле id совпадает с переданным task_id. Далее идёт создание словаря контекста, где ключ task станет переменной, которую можно использовать внутри HTML-шаблона для вывода данных задачи. И наконец последняя строка - возврат ответа пользователю. Функция объединяет HTML-шаблон tasks/task_detail.html с данными из словаря context и отправляет готовую страницу в браузер.

```
86 def task_detail(request, task_id):
87     """Страница отдельной задачи"""
88     task = Task.objects.get(id=task_id)
89     context = {
90         'task': task,
91     }
92     return render(request, 'tasks/task_detail.html', context)
```

Рис. 51 Функция-обработчик task_detail

Теперь создаём html страницу для вывода информации о задаче:

```
tasks > templates > tasks > task_detail.html > html > body > p
1  <!DOCTYPE html>
2  <html>
3  <head>
4  |   <title>{{ task.title }}</title>
5  </head>
6  <body>
7  |   <a href="/">← На главную</a>
8
9  |   <h1>{{ task.title }}</h1>
10
11 |   {% if task.completed %}
12 |   |   <p>✅ Статус: Выполнено</p>
13 |   {% else %}
14 |   |   <p>⌚ Статус: В процессе</p>
15 |   {% endif %}
16
17 |   <p>📅 Создано: {{ task.created_at|date:"d.m.Y H:i" }}</p>
18
19 |   {% if task.description %}
20 |   |   <h2>Описание:</h2>
21 |   |   <p>{{ task.description }}</p>
22 |   {% endif %}
23 </body>
24 </html>
```

Рис. 52 Страница task_detail.html

А также меняем на домашней странице заголовок на заголовок с ссылкой.

```
<h3>
|   <a href="/task/{{ task.id }}/">{{ task.title }}</a>
```

Рис. 53 Заголовок с ссылкой

Последним шагом добавляем паттерн в url.py:

```
path('task/<int:task_id>/', views.task_detail, name='task_detail')
]
```

Рис. 53 Изменения в url.py

Результат:

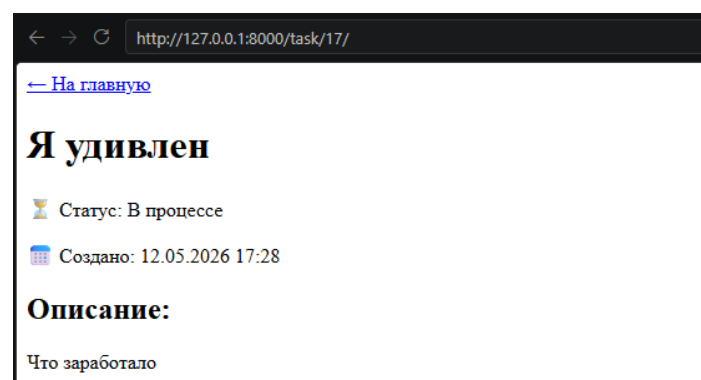


Рис. 54 Результат

URL через name

Проблема: сейчас мы пишем `/task/{{ task.id }}/` — если изменим URL в `urls.py`, ссылки сломаются.

Решение: будем использовать `{% url %}`, сделаем изменения в `home.html`:

```
<a href="{% url 'task_detail' task.id %}">{{ task.title }}</a>
{% if task.completed %}
```

Рис. 55 Изменения в `home.html`

Таким же образом заменим все остальные формы и запросы:

```
<form action="{% url 'addTask' %}" method="post">{% csrf_token %}
<form action="{% url 'complete_task' %}" method="post">
<form action="{% url 'delete_task' %}" method="post">
<form action="{% url 'edit_task' %}" method="post">
```

Рис. 56 Изменение маршрута через имя

В конце нужно удостовериться, что все имена маршрутов совпадают:

```
urlpatterns = [
    path('', views.home, name='home'),
    path('hello/<str:name>/', views.hello, name='hello'),
    path('user/<str:username>/<int:age>/', views.user_profile, name='profile'),
    path('addTask/', views.addTask, name='add_task'),
    path('tasks/pending/', views.pending_tasks, name='pending'),
    path('tasks/completed/', views.completed_tasks, name='completed'),
    path('deleteTask/', views.delete_task, name='delete_task'),
    path('completeTask/', views.complete_task, name='complete_task'),
    path('editTask/', views.edit_task, name='edit_task'),
    path('task/<int:task_id>/', views.task_detail, name='task_detail')
]
```

Рис. 57 Проверка паттернов маршрутов

Статические файлы (CSS)

Само собой оформление html страниц требует использование css стилей. Создадим в `tasks` место для их хранения:

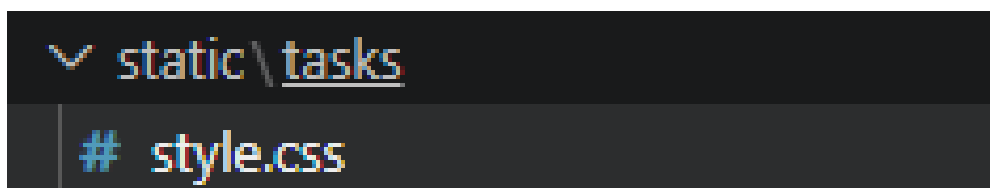


Рис. 58 Расположение css стилей в проекте

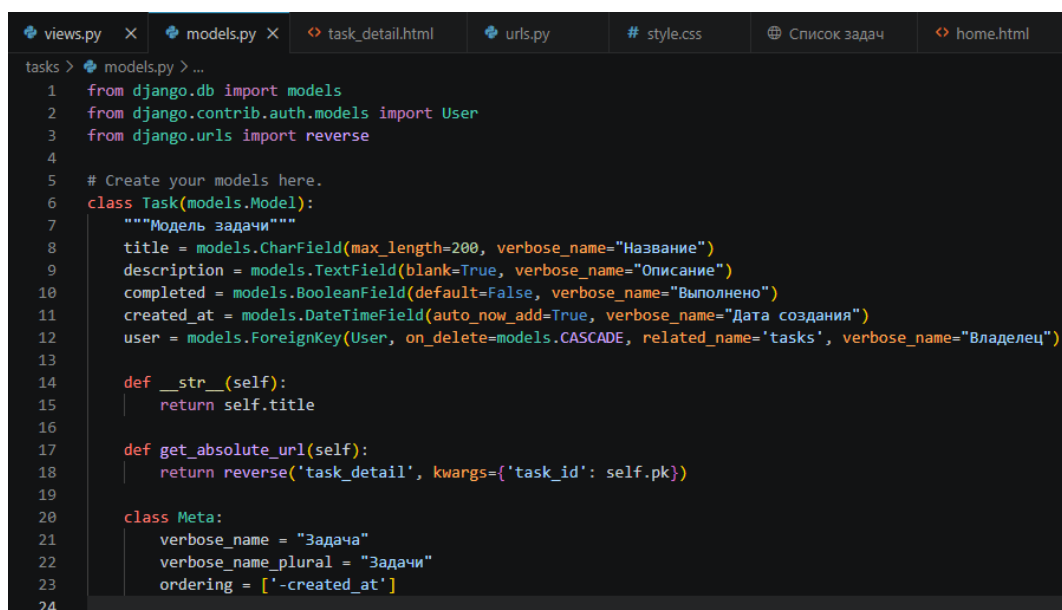
Заполняем файл стилями и подключаем их в страницах:

```
<title>Список задач</title>
<link rel="stylesheet" href="{% static 'tasks/style.css' %}">
<title>{{ task.title }}</title>
<link rel="stylesheet" href="{% static 'tasks/style.css' %}">
```

Рис. 59 Подключение CSS стилей

Авторизация. Связь задач с пользователем

Django уже имеет встроенную модель User. Нам нужно связать Task с User.

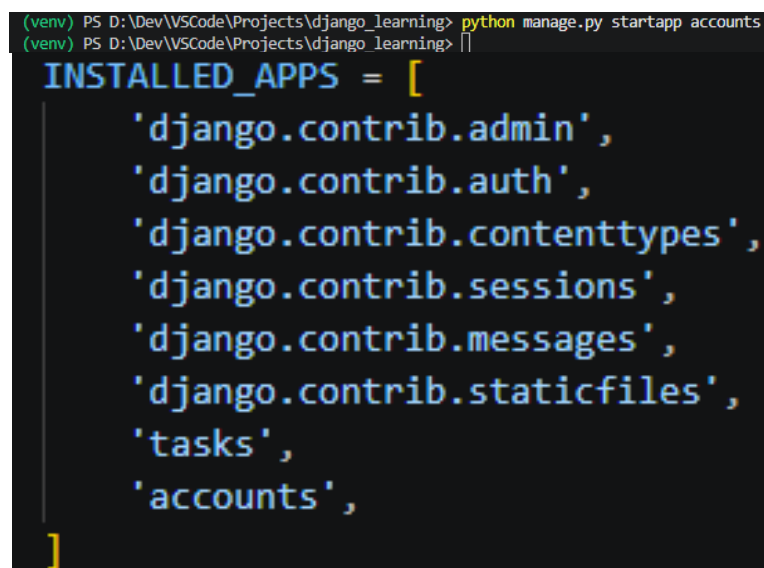


```
tasks > models.py > ...
1 from django.db import models
2 from django.contrib.auth.models import User
3 from django.urls import reverse
4
5 # Create your models here.
6 class Task(models.Model):
7     """Модель задачи"""
8     title = models.CharField(max_length=200, verbose_name="Название")
9     description = models.TextField(blank=True, verbose_name="Описание")
10    completed = models.BooleanField(default=False, verbose_name="Выполнено")
11    created_at = models.DateTimeField(auto_now_add=True, verbose_name="Дата создания")
12    user = models.ForeignKey(User, on_delete=models.CASCADE, related_name='tasks', verbose_name="Владелец")
13
14    def __str__(self):
15        return self.title
16
17    def get_absolute_url(self):
18        return reverse('task_detail', kwargs={'task_id': self.pk})
19
20    class Meta:
21        verbose_name = "Задача"
22        verbose_name_plural = "Задачи"
23        ordering = ['-created_at']
24
```

Рис. 60 Связь задачи с конкретным пользователем.

После применяем миграции. Теперь реализуем регистрацию и авторизацию.

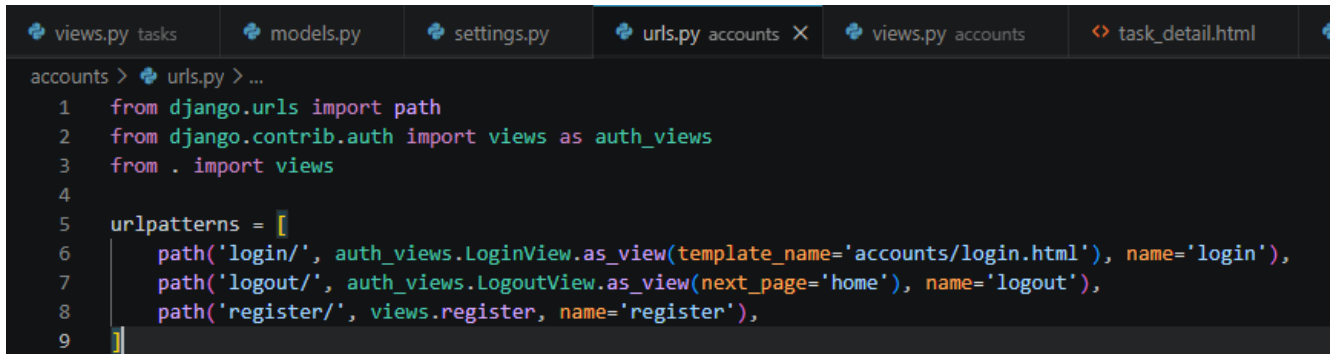
Для этого создадим приложение для пользователей.



```
(venv) PS D:\Dev\VSCode\Projects\django_learning> python manage.py startapp accounts
(venv) PS D:\Dev\VSCode\Projects\django_learning>
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'tasks',
    'accounts',
]
```

Рис. 61 Создание и регистрация нового приложения

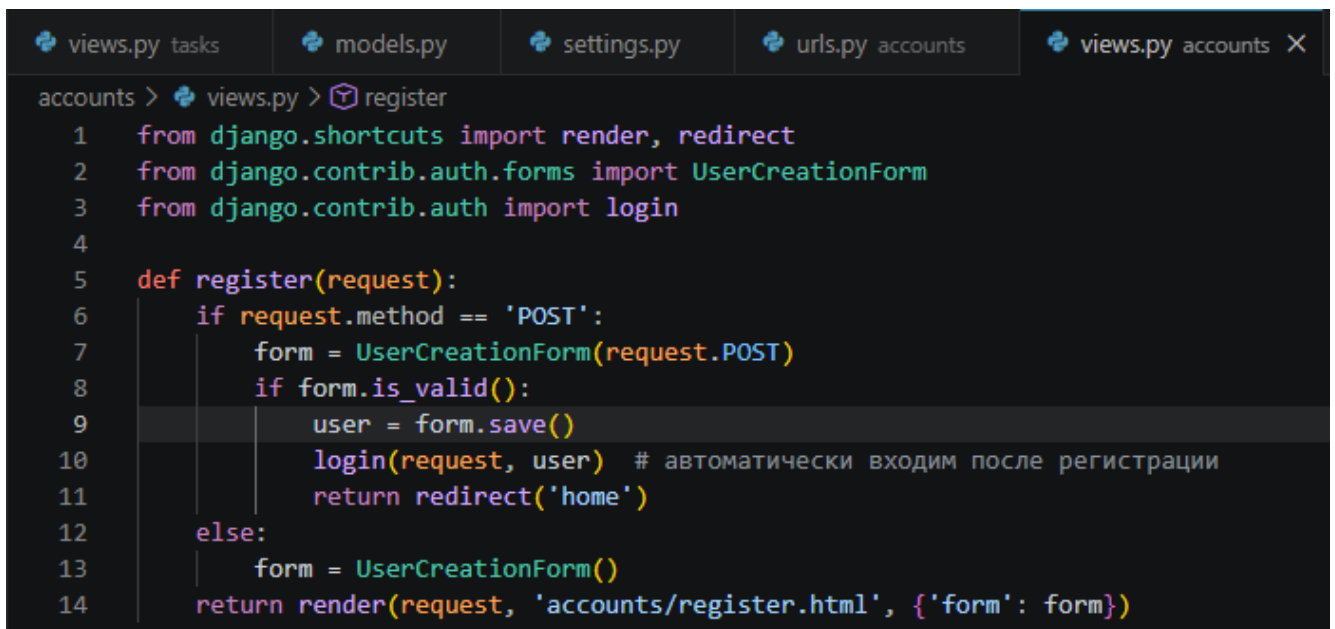
Создаём в новом приложении urls и пишем паттерны для будущих маршрутов.



```
accounts > urls.py > ...
1 from django.urls import path
2 from django.contrib.auth import views as auth_views
3 from . import views
4
5 urlpatterns = [
6     path('login/', auth_views.LoginView.as_view(template_name='accounts/login.html'), name='login'),
7     path('logout/', auth_views.LogoutView.as_view(next_page='home'), name='logout'),
8     path('register/', views.register, name='register'),
9 ]
```

Рис. 62 Urls.py приложения accounts

Создаём обработчики во view.py.



```
accounts > views.py > register
1 from django.shortcuts import render, redirect
2 from django.contrib.auth.forms import UserCreationForm
3 from django.contrib.auth import login
4
5 def register(request):
6     if request.method == 'POST':
7         form = UserCreationForm(request.POST)
8         if form.is_valid():
9             user = form.save()
10            login(request, user) # автоматически входим после регистрации
11            return redirect('home')
12     else:
13         form = UserCreationForm()
14     return render(request, 'accounts/register.html', {'form': form})
```

Рис. 63 Views.py приложения accounts

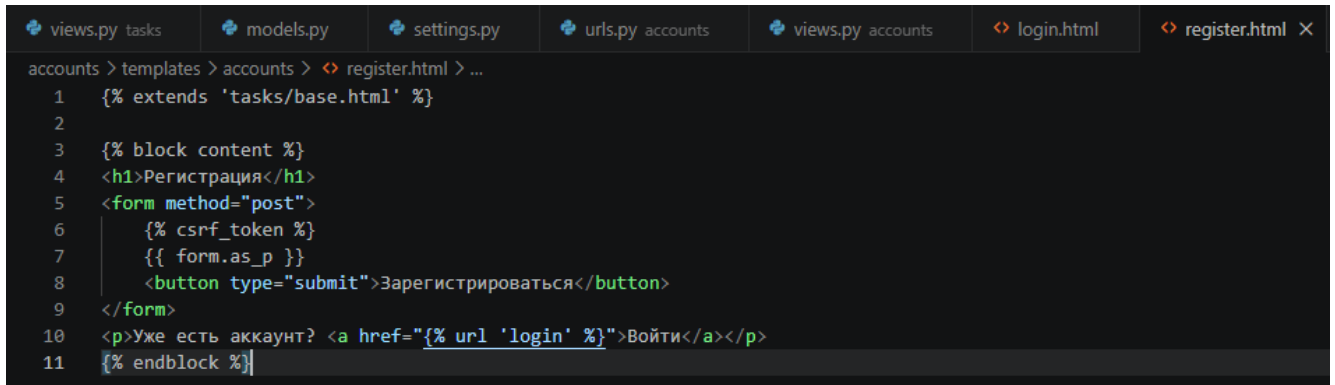
Создаём папку с шаблонами и заполняем шаблон login.



```
accounts > templates > accounts > login.html > ...
1 {% extends 'tasks/base.html' %}
2
3 {% block content %}
4     <h1>Вход</h1>
5     <form method="post">
6         {% csrf_token %}
7         {{ form.as_p }}
8         <button type="submit">Войти</button>
9     </form>
10    <p>Нет аккаунта? <a href="{% url 'register' %}">Зарегистрируйся</a></p>
11    {% endblock %}
```

Рис. 64 Содержание шаблона login.html

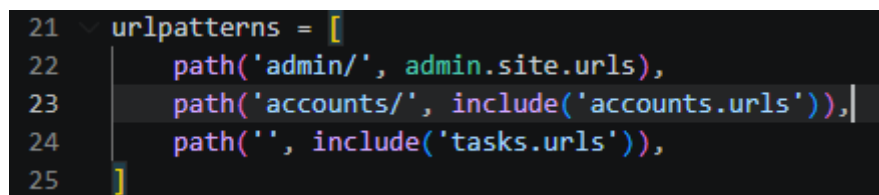
А также заполняем шаблон register.



```
accounts > templates > accounts > <> register.html > ...
1  {% extends 'tasks/base.html' %}
2
3  {% block content %}
4  <h1>Регистрация</h1>
5  <form method="post">
6      {% csrf_token %}
7      {{ form.as_p }}
8      <button type="submit">Зарегистрироваться</button>
9  </form>
10 <p>Уже есть аккаунт? <a href="{% url 'login' %}">Войти</a></p>
11 {% endblock %}
```

Рис. 65 Содержание шаблона register.html

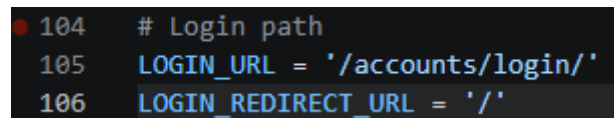
Подключаем accounts/urls.py в главный mytodo/urls.py:



```
21 urlpatterns = [
22     path('admin/', admin.site.urls),
23     path('accounts/', include('accounts.urls')),
24     path('', include('tasks.urls')),
25 ]
```

Рис. 66 Подключение в главный urls.py

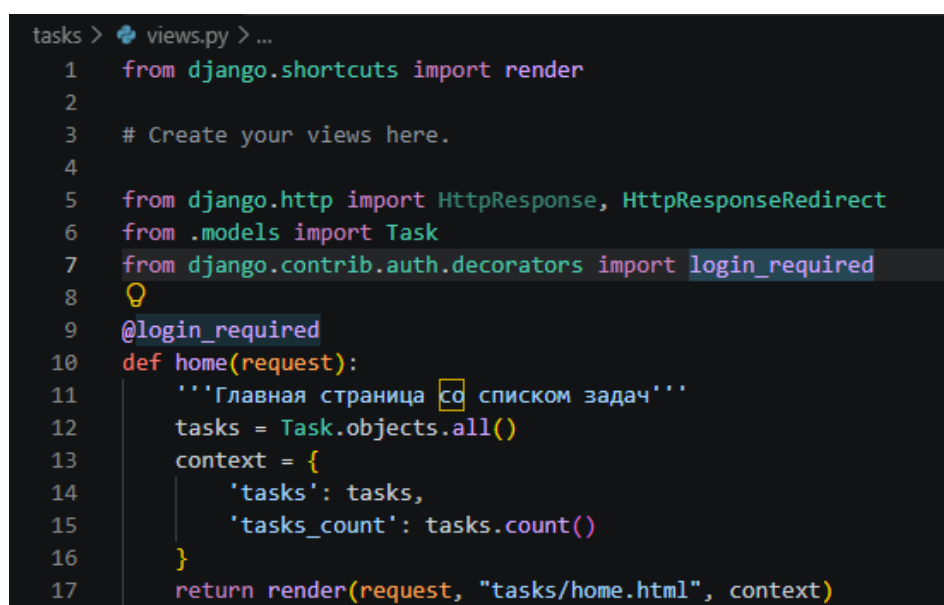
Настраиваем пути для логина в settings.py.



```
104 # Login path
105 LOGIN_URL = '/accounts/login/'
106 LOGIN_REDIRECT_URL = '/'
```

Рис. 67 Настройка путей

В tasks/view.py добавим импорт и @login_required перед каждой view, которая требует авторизации:



```
tasks > views.py > ...
1  from django.shortcuts import render
2
3  # Create your views here.
4
5  from django.http import HttpResponseRedirect, HttpResponseRedirect
6  from .models import Task
7  from django.contrib.auth.decorators import login_required
8
9  @login_required
10 def home(request):
11     '''Главная страница со списком задач'''
12     tasks = Task.objects.all()
13     context = {
14         'tasks': tasks,
15         'tasks_count': tasks.count()
16     }
17     return render(request, "tasks/home.html", context)
```

Рис. 68 Импорт и использование @login_required

В конце останется немного изменить обработчики view, для проверки пользователя и по причине добавления в модель нового элемента user. Авторизация готова, результат:

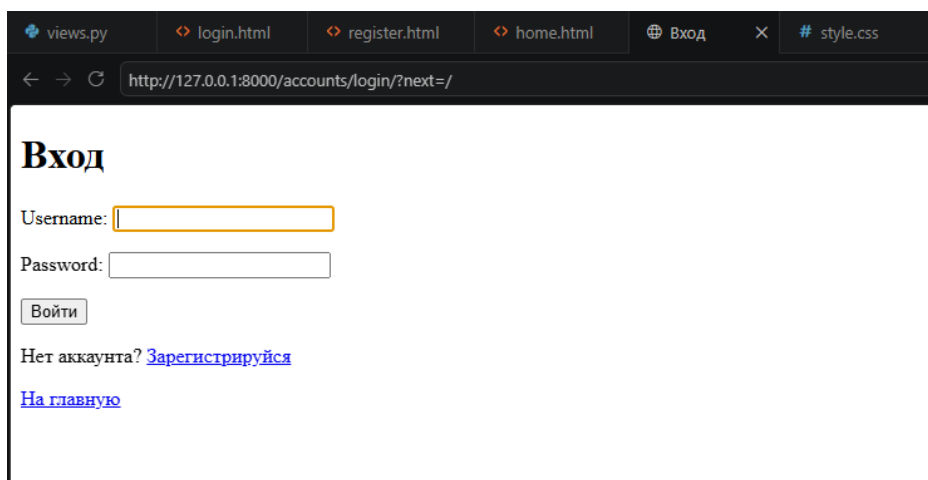


Рис. 69 Страница входа

Этап улучшения

В конце сделаем улучшения, которые приведут наш сайт к нормальному виду. Первым будет создание профиля пользователя, изображенный на рисунке 70.

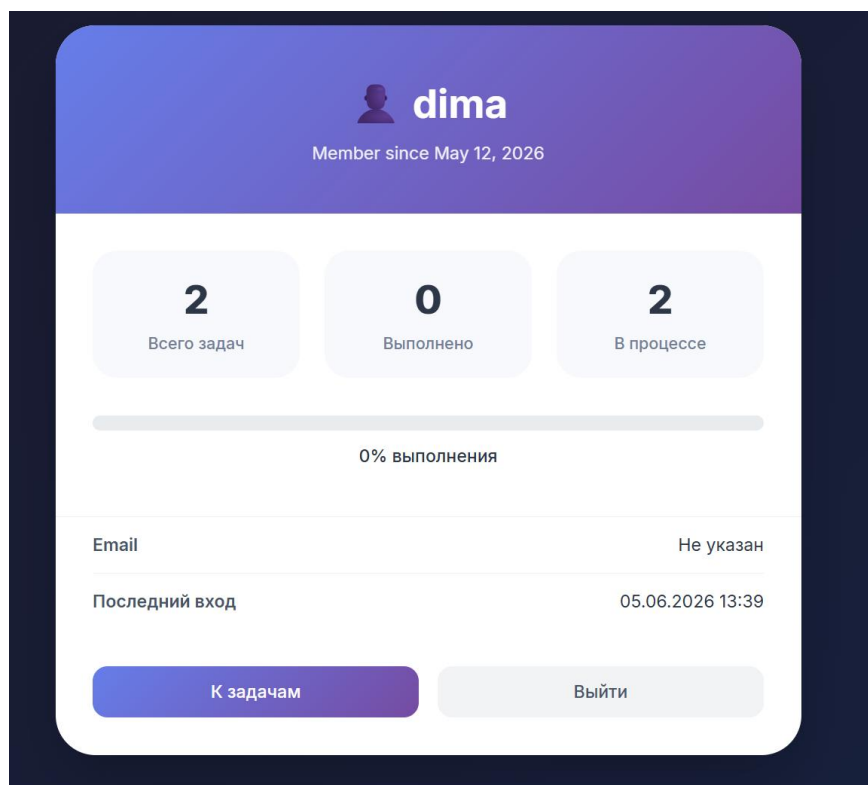


Рис. 70 Профиль пользователя

Вторым, улучшив визуальную составляющую и украсим интерфейс, выведем стили в отдельный файл и переведем предупреждения и ошибки при регистрации. На рисунке 71 представлены финальные результаты.

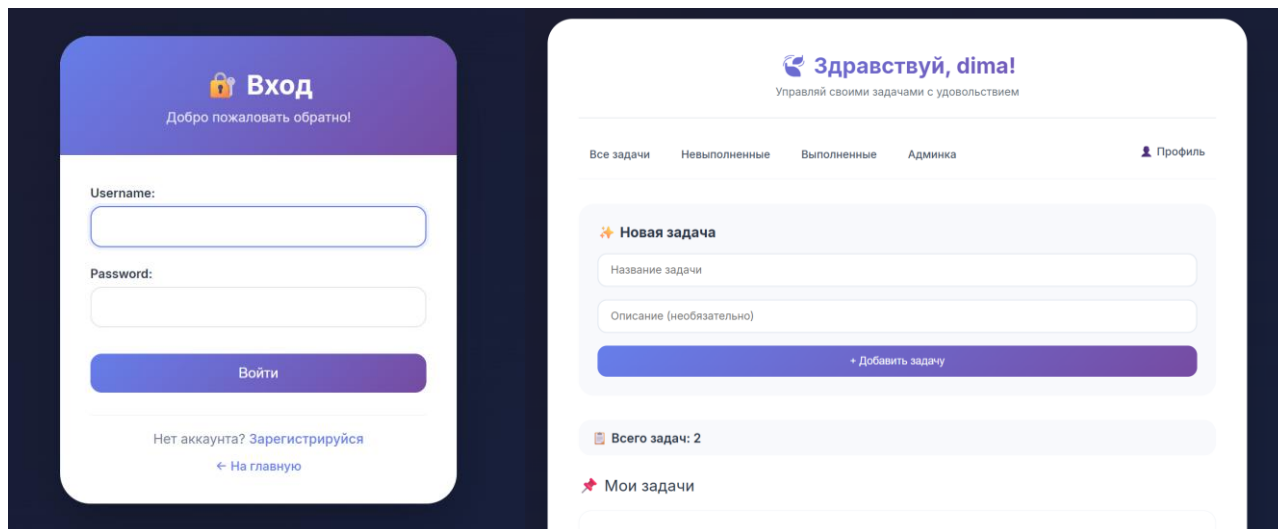


Рис. 71 Результаты

ИТОГИ

Было разработано полноценное веб-приложение на Django — Todo-менеджер с авторизацией. Проект включает в себя всё, что нужно для современного веб-приложения:

- Регистрация и авторизация пользователей;
- Создание, просмотр, редактирование и удаление задач;
- Отметка о выполнении задачи;
- Фильтрация задач (все, выполненные, невыполненные);
- Личный профиль пользователя со статистикой выполнения задач;
- Возможность добавить и изменить email в профиле;
- Адаптивный и современный дизайн;
- Полная документация проекта с пояснениями и скриншотами.

Что было освоено в процессе

Тема	Что научился делать
Django основы	Создавать проекты и приложения, настраивать маршруты
Модели	Описывать структуру базы данных через классы, применять миграции
CRUD операции	Создавать, читать, обновлять и удалять записи в базе данных
Шаблоны Django	Верстать HTML-страницы с переменными, условиями и циклами
Статические файлы	Подключать CSS и JS к проекту

Авторизация	Регистрация, вход, выход, защита страниц от неавторизованных пользователей
Связь с пользователем	Привязывать данные к конкретному пользователю
Фильтрация данных	Показывать только нужные записи из базы данных
Классовые View	Использовать ListView, DetailView, CreateView, UpdateView, DeleteView
Git и GitHub	Контроль версий, создание репозитория, коммиты, push
Документация	Оформлять код и писать понятные объяснения с примерами